



HIGH-LEVEL DESIGN DOCUMENT

A novel method for video manipulation and compilation using
Big Data Technologies

UE21CS320A – Capstone Project Phase – 1

Submitted by:

NAME	SRN
Abhishek V Sagarnal	PES2UG21CS023
Divya D	PES2UG21CS165
Pradyun P Rao	PES2UG21CS378
Sarthak S Kumar	PES2UG21CS482

Under the guidance of
Dr. Chandrashekhar P Chavan
PES University

January - May 2024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

TABLE OF CONTENTS

1. Introduction	4
2. Current System	4
3. Design Considerations	4
3.1 Design Goals	4
3.2 Architecture Choices	5
3.3 Constraints, Assumptions and Dependencies	5
4. High Level System Design	6
5. Design Description	8
5.1 Master Class Diagram	9
5.2 Reusability Considerations	9
6. User Interface Diagrams	10
7. Report Layouts	11
8. External Interfaces	11
9. Help	12
10. Design Details	13
12.1. Novelty and Innovativeness	
12.2. Reliability	
12.3. Performance	
12.4. Reusability	
12.5. Application Compatibility	

Appendix A: Definitions, Acronyms and Abbreviations	13
Appendix B: References	14
Appendix C: Record of Change History	14
Appendix D: Traceability Matrix	15

Note:

Section – 1 & Section 2	Common for Product-Based and Research Projects
Section 3 to Section 11	High-Level Design for Product-Based Projects.
Section 12	High-Level Design for Research Projects.
Appendix	Provide details appropriately

1. Introduction

This document outlines the high-level design of a video processing solution tailored to efficiently encode, analyze, manipulate, and compile video content. It aligns with the requirements specified in the Project Requirement Specifications (PRS) document, ensuring functionality and performance.

2. Current System [if applicable]

Utilizing Spark for distributed processing and FFmpeg for video encoding, coupled with OpenCV for frame manipulation such as splitting and resizing.

Challenges:

- FFmpeg's limitations in distributed environments necessitate exploring alternative solutions or potentially developing a custom solution.
- Optimization of GPU utilization by considering leveraging Nvidia CUDA capabilities to enhance performance, including building OpenCV for CUDA to tap into GPU acceleration.

3. Design Considerations

3.1. Design Goals

The new approach utilizes Spark across a multi-node cluster for distributed task execution, employing dynamic partitioning techniques and configuration experimentation to optimize resource utilization and enhance throughput, aiming for improved performance and scalability.

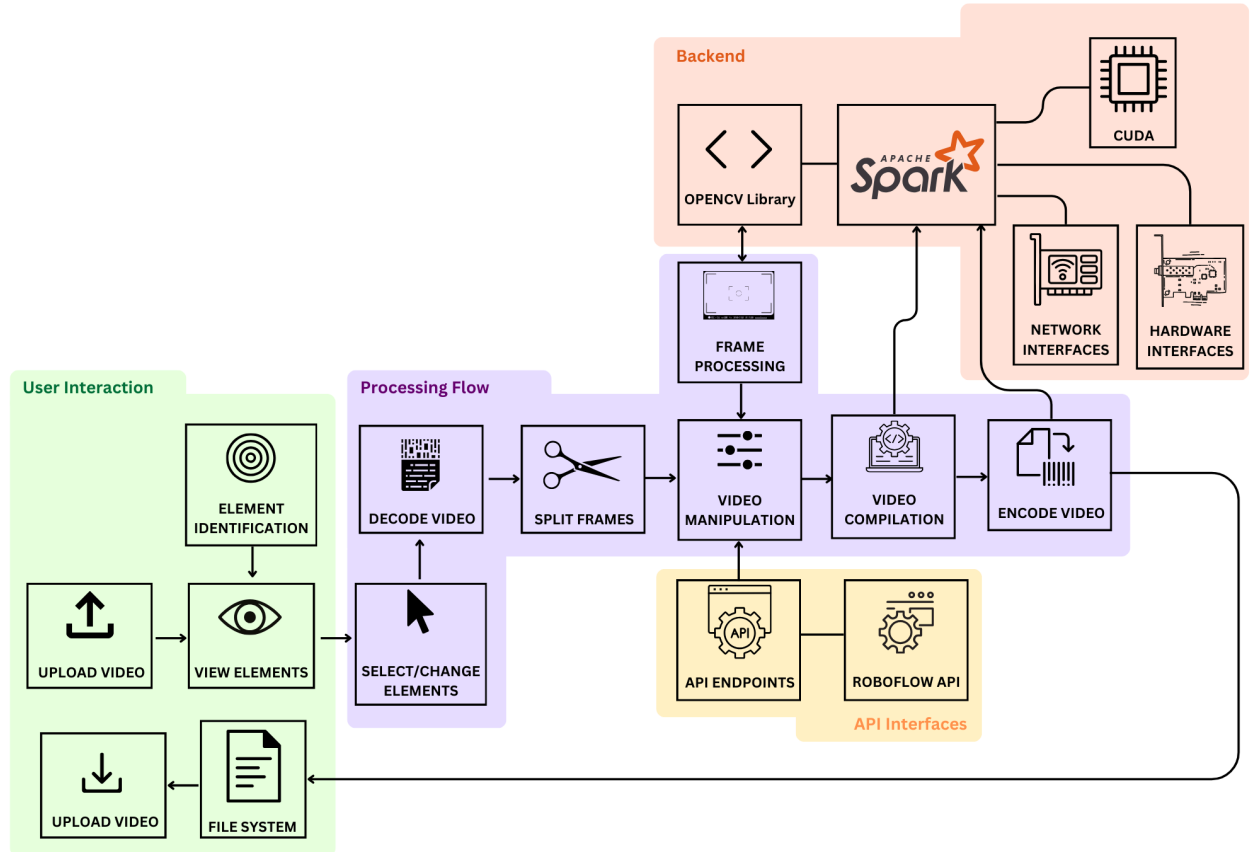
Benefits of new approach:

- Distributed task execution with Spark improves processing capabilities.
- Dynamic partitioning and configuration experimentation optimize resource utilization and throughput.
- Addresses FFmpeg's limitations in distributed environments.
- Leverages GPU acceleration for enhanced performance.
- Potential for faster processing times and improved efficiency.

The quality of service characteristics include:

- Availability: Ensuring consistent accessibility with redundancy and failover mechanisms.
- Speed and Performance: Optimizing processing speed and efficiency.
- Scalability: Scaling seamlessly to accommodate growing demand.
- Reliability: Providing consistent and accurate results with fault tolerance mechanisms.
- Usability: Offering a user-friendly interface and intuitive workflows.

3.2. Architecture Choices



Chosen Architecture:

- The chosen architecture for the video processing system is leveraging Big Data technologies like Apache Spark for parallel processing.
- The system integrates CUDA capabilities with Spark for enhanced processing efficiency.

Pros:

- **Scalability:** Spark scales efficiently to handle large video volumes across clusters.
- **Fault Tolerance:** Built-in mechanisms ensure reliability and continuous operation.
- **Compatibility:** Supports various data sources and formats for flexibility.
- **Performance:** In-memory processing enhances speed and efficiency.

Cons:

- **Deployment Complexity:** Setup and deployment across clusters can be intricate.
- **Dynamic Partitioning Challenges:** Optimizing dynamic partitioning techniques may be complex.
- **Learning Curve:** Requires familiarity with distributed computing concepts and setting up a cluster.
- **Resource Intensive:** Demands significant hardware resources for optimal operation.

3.3. Constraints, Assumptions and Dependencies**Assumptions:**

- **Resource Availability:** Assumes availability of resources for optimized framework setup and hardware efficiency.
- **End-user Environment:** Assumes the minimization of processing overhead and maximization of resource utilization for a seamless user experience.

Dependencies:

- **Big Data Technologies:** Dependency on Apache Spark for parallel processing of video frames, ensuring efficient data handling and analysis.

Constraints:

- **Efficient Video Analysis:** Constraints imposed by the need for efficient video analysis, relying on tools like OpenCV and Spark RAPIDS library for optimized processing.

Performance:

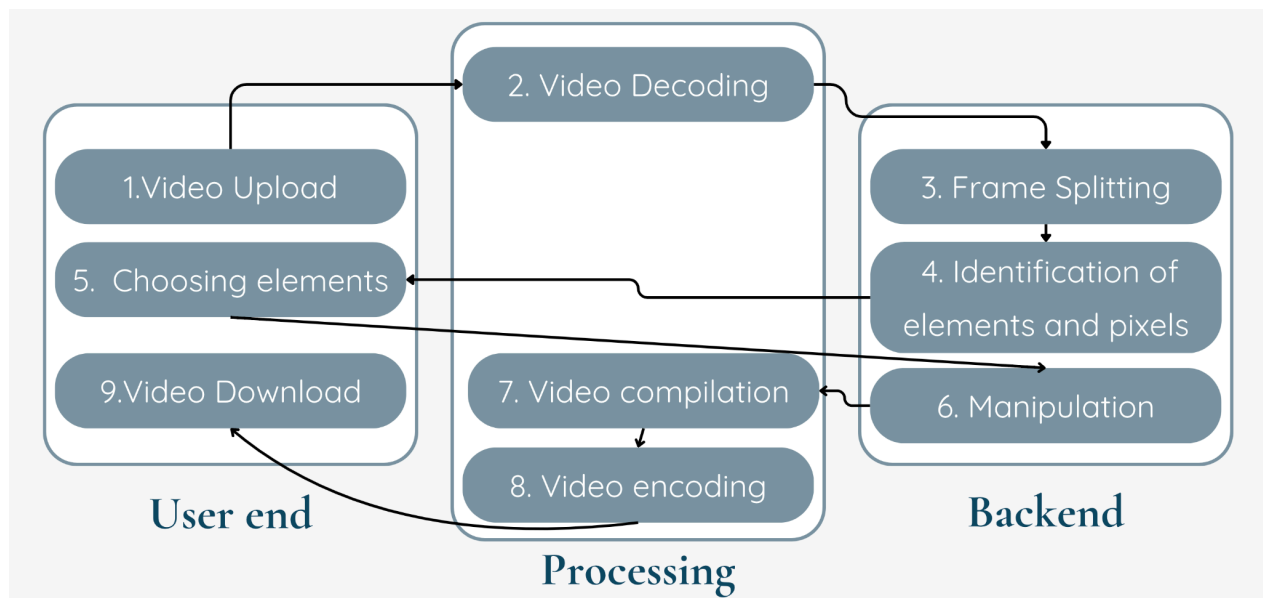
- **Dynamic Partitioning:** Performance issues are addressed through dynamic partitioning techniques and configurations to enhance processing efficiency.

4. High Level System Design**System Design Components:**

- **Logical User Groups:** Users interacting with the video processing system for tasks like frame manipulation and video analysis.
- **Application Components:** Components like Apache Spark, OpenCV, and machine learning frameworks like CUDA are integrated for efficient video processing.
- **Data Components:** The system processes large video volumes by splitting videos into frames and analyzing them in parallel across a distributed framework like Spark.
- **Interfacing Systems:** The system interfaces with components for encoding, decoding, frame manipulation, and compilation to meet project requirements.

Collaboration and Interaction Between Major Components:

- The video processing system integrates Apache Spark with CUDA capabilities for enhanced processing efficiency.
- Components like Spark, FFmpeg, and OpenCV work together for frame splitting and resizing.
- The system leverages Big Data technologies to handle large video volumes efficiently through distributed processing.
- Machine learning frameworks and CUDA are integrated for enhanced video processing capabilities.



Workflow design:

- **Video Decoding:** Decode the input video file to extract raw video data.
- **Splitting of Video into Frames:** Divide the decoded video stream into individual frames for further processing.
- **Pixel Selection:** Analyze and select specific pixels within each frame based on defined criteria or algorithms.
- **Video Manipulation:** Apply desired manipulations or transformations to the selected pixels or entire frames.
- **Compilation of Frames:** Reassemble the manipulated frames into a cohesive video sequence, ensuring proper sequencing and timing.
- **Video Encoding:** Encode the compiled video sequence into the desired output format, resolution, and quality

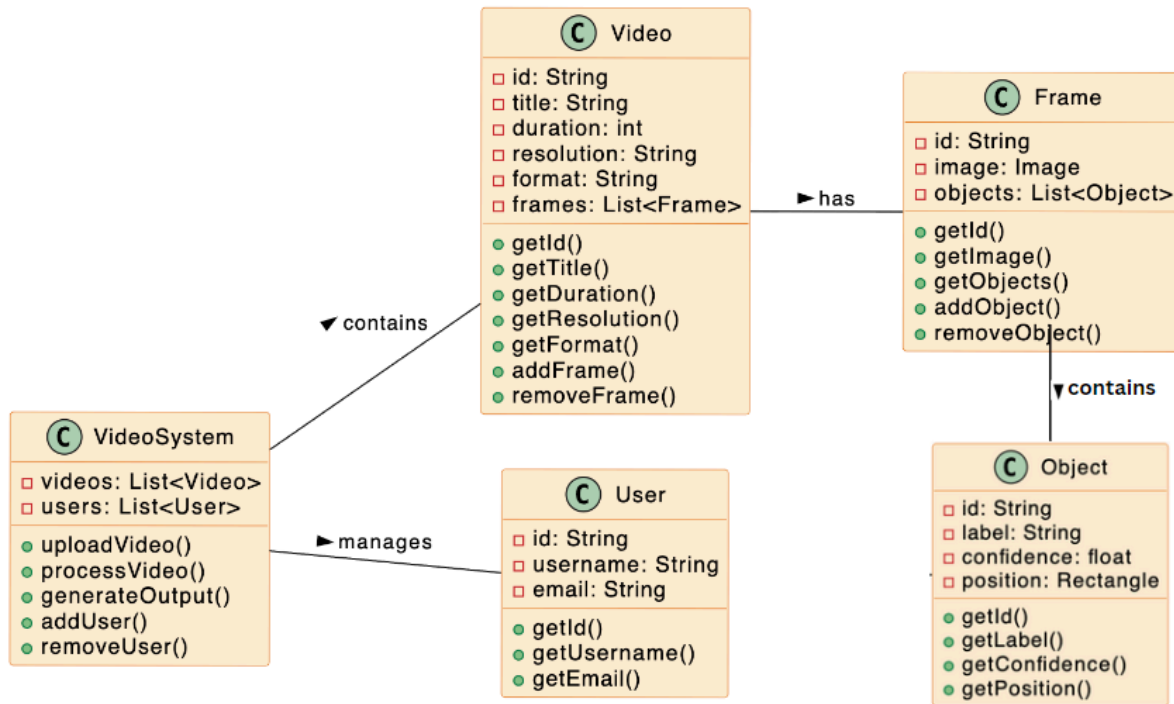
Technical specifications:

- Utilize Apache Spark for distributed processing of video data.
- Incorporate machine learning frameworks like TensorFlow, PyTorch, and CUDA for enhanced processing capabilities.
- Design fault tolerance mechanisms for uninterrupted operation.
- Ensure compatibility with evolving platforms for reusability.
- Implement components for encoding, decoding, frame manipulation, and compilation to meet project requirements.
- Optimize algorithms and processing pipelines for high throughput and low latency.

5. Design Description**Utilized Modules in the Design**

- 1) **Video Processing Module:** Optimizes video files by accepting input and providing output in desired format and quality.
- 2) **Frame Analysis Module:** Analyzes frames to detect objects/logos, sending results to other modules.
- 3) **Video Compilation Module:** Creates cohesive video sequences from processed frames.
- 4) **User Interface Module:** Facilitates user interaction, input, feedback, and result display.
- 5) **Authentication Module:** Manages user authentication, authorization, and access control.

5.1. Master Class Diagram



5.2. Reusability Considerations

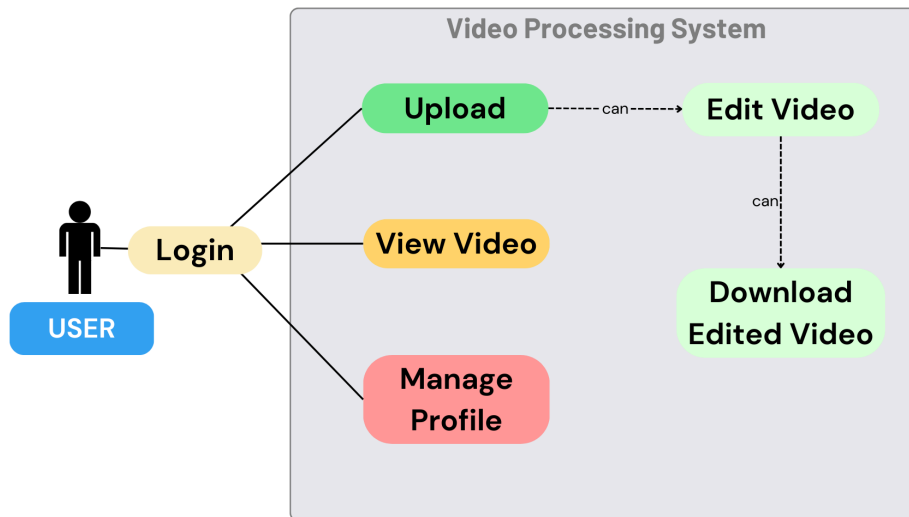
Utilizing Available Reusable Components:

- Video Processing Library: Incorporate OpenCV or FFmpeg for encoding, decoding, and frame manipulation.
- Authentication Framework: Integrate OAuth or JWT for user authentication and authorization.

Components Built for Reuse within the Project:

- Custom Video Processing Algorithms: Develop tailored algorithms for tasks like object detection or video stabilization.
- User Interface Components: Design reusable UI elements such as custom video player widgets and interactive controls.
- Data Access Layer: Implement a reusable data access layer with database components and ORM frameworks.

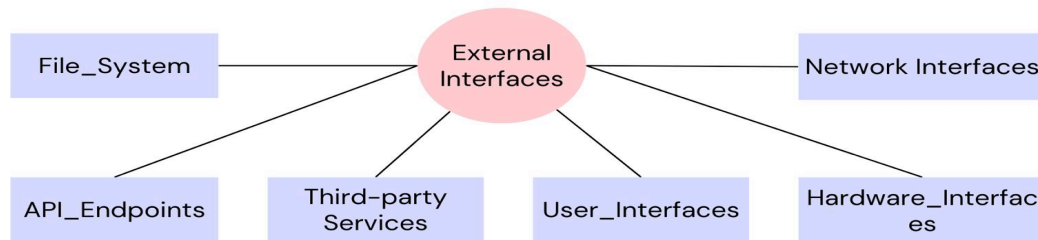
6. User Interface Diagrams



7. Report Layouts

The report layouts for the high-level design document include an introduction providing an overview of the document's purpose, followed by a section on the current system if applicable. Design considerations are outlined, including goals, architecture choices, and constraints. The high-level system design section describes user groups, components, and interfaces, while the design description delves into details of key modules. Diagrams such as Master class diagram, User interface diagram are included, along with discussions on reusability considerations and external interfaces. The document also covers packaging and deployment details, support and documentation in the help section, and design specifics like novelty, performance, and security. Finally, appendices provide definitions, references, change history, and a traceability matrix for requirements.

8. External Interfaces



9. Help

- Online and context-sensitive help integrated within the system interface.
- Assistance available on-demand to aid users in navigating and utilizing system features effectively.
- Provision of comprehensive documentation, including User Manual and Technical Manual.
- User Manual offering detailed instructions on system functionality and usage.
- Technical Manual providing in-depth information on system configuration and troubleshooting procedures.

10. Design Details

12.1. Novelty and Innovativeness:

Continuously innovate with advanced algorithms for object detection and video processing to stay ahead.

12.2. Reliability:

Design fault tolerance mechanisms for uninterrupted operation.

12.3. Performance:

Optimize algorithms and processing pipelines for high throughput and low latency.

12.4. Reusability:

Design components to be agnostic to specific environments, facilitating reuse in different projects.

12.5. Application Compatibility:

Ensuring to maintain compatibility with evolving platforms.

Appendix A: Definitions, Acronyms and Abbreviations

Term	Definition
CDN	Content delivery network
CI/CD	Continuous integration and continuous deployment
GPU	Graphics processing unit
Metadata	Data that provides information about other data
OpenCV	Open-source computer vision library
Spark	A distributed processing framework
FFmpeg	A free and open-source software library for handling multimedia files

Appendix B: References

<https://link.springer.com/article/10.1007/s00521-018-03983-z>

<https://ieeexplore.ieee.org/abstract/document/7874183>

<https://www.scirp.org/journal/paperinformation?paperid=113330>

<https://arxiv.org/abs/1805.07339> <https://spark.apache.org/> <https://hadoop.apache.org/>
https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html

<https://www.techtarget.com/searchstorage/essentialguide/Choosing-storage-for-streaming-large-files-in-big-data-sets>

Appendix C: Record of Change History

Table C-1: Record of Change History

The following table provides a record of changes that have been made to the High-Level Design (HLD) document since its initial version.

Date	Document Version No.	Change Description	Reason for Change
April 8, 2024	1.0	Initial version of the HLD document	Created to document the high-level design of the proposed video processing system.
April 11, 2024	1.1	Updated the HLD document to reflect the latest changes to the design.	Incorporated feedback from initial reviews and made minor modifications to the design to improve efficiency and scalability.
April 15, 2024	1.2	Updated the HLD document to add a new section on security considerations.	Added a discussion of the security requirements for the video processing system and how they will be addressed in the design.
April 18, 2024	1.3	Updated the HLD document to reflect the final design decisions.	Finalized the design of the video processing system and documented all of the key decisions that were made.

Appendix D: Traceability Matrix

Requirement ID	Functional Requirement	DESIGN / HLD Reference Section No. and Name
1.1	The system shall process video manipulation tasks efficiently...	5.1
1.2	The system should scale seamlessly with increasing demand...	5.2
1.3	Optimize resource utilization, including CPU, memory, and storage...	5.3
2.1	Implement safeguards to prevent accidental deletion or corruption...	6.1
2.2	Design robust error handling mechanisms to detect and recover...	6.2
2.3	Manage system resources effectively to prevent resource exhaustion...	6.3
3.1	Implement role-based access control (RBAC) mechanisms...	7.1
3.2	Require user authentication before granting access to the system...	7.2
3.3	Maintain detailed audit logs of user activities...	7.3